

SECTION 6

EXCEPTION PROCESSING

This section describes how the MC68881/MC68882 (FPCP) and the main processor handle exceptional conditions during the processing of floating-point instructions. These exceptional conditions may be detected internally by the FPCP, internally by the main processor, or externally by the main processor.

The MC68020/MC68030 (MPU) processes exceptions by treating any coprocessor in an M68000 system as an extension to the main processor; the fact that a coprocessor is separate from the main processor is transparent to the programmer. Thus, the exception processing for all coprocessors in a system is coordinated by the main processor in a manner that is consistent across all exception types, whether detected during the execution of an instruction native to the main processor or during a coprocessor instruction.

The processing of an exception detected during the execution of an FPCP instruction involves the following basic steps:

1. Detect the exception
2. Determine the exception vector number and report the exception to the main processor (if detected by the FPCP)
3. Change processing states if needed (user to supervisor)
4. Save the old context of the main processor (performed automatically by the MPU)
5. Load a new context from the address contained in the exception vector table
6. Execute the exception handler
7. Return to the previous context

The first two steps involve slightly different operations for exceptions detected by the main processor and those detected by the FPCP, but the manner in which these operations are performed is consistent with noncoprocessor related exceptions. The major difference in the processing of exceptions detected by the FPCP and the main processor is the point at which exception processing starts. For all main-processor-detected exceptions and some coprocessor-detected exceptions, processing for the exception begins during the execution of the coprocessor instruction by the main processor. However, for many of the coprocessor-detected exceptions, processing for the exception does not begin until after the main processor completes execution of the offending instruction and attempts execution of a new floating-point instruction. The manner of handling this type exception supports a sequential instruction programming model.

The action of the processor during step 7 depends upon the type of exception that was previously taken. When the exception handler completes execution, a return from exception (RTE) instruction is executed, and the previously interrupted program resumes execution at one of the following points:

1. The beginning of the instruction that was pre-empted by an exception detected by or reported to the MPU (pre-instruction exception)
2. The point where the exception occurred during the execution of an instruction (mid-instruction exception)



The beginning of the instruction immediately following the instruction that caused or detected the exception (post-instruction exception). Note that neither the MC68881 nor the MC68882 reports the post-instruction exception.

The following paragraphs describe the causes of various coprocessor-related exceptions and how they are handled by the FPCP and the main processor. Throughout this discussion, the main processor is assumed to be an MC68020 or MC68030, although any other processor can be programmed to emulate the M68000 Family coprocessor interface that is implemented on the MPU.

6.1 COPROCESSOR-DETECTED EXCEPTIONS

Coprocessor-detected exceptions fall into two categories: those related to communications with the main processor (F-line traps and protocol violations) and those related to the execution of floating-point instructions (computational errors such as divide by zero, or instructions designed to cause a trap such as the FTRAPcc instruction). The protocol for handling each of these exception types is described in detail in this section.

6

The main processor coordinates all exception processing. Therefore, when the FPCP detects an exception, it cannot always force exception processing immediately but must wait until the main processor is ready to start exception processing. The main processor is always prepared to process an exception whenever it reads the response coprocessor interface register (CIR). For the MC68881 and, in most cases, for the MC68882, if a coprocessor-detected exception occurs during the calculation phase of an instruction, it is held pending within the FPCP until the next write to the command or condition coprocessor interface register (CIR). Then, instead of returning the first primitive of the dialog for the new instruction, the FPCP returns the take pre-instruction exception primitive to start exception processing for the offending instruction. (For the MC68881, the offending instruction is always the previous floating-point instruction, since no multiple floating-point concurrency is allowed; for the MC68882, the offending instruction may not necessarily be the previous instruction.)

The FPCP may also report an exception after writing an operand to memory. In this case, a take mid-instruction exception primitive is issued after the operand is stored in memory (if a conversion error occurred). The mid-instruction exception allows the exception handler to more easily determine the address of the exceptional operand, since the MC68020 includes the results of the effective address calculation for the destination operand in the mid-instruction stack frame (the long word at offset +\$10).

It is possible for the MC68882 to report a mid-instruction exception as a result of an exception created by a previous instruction. This occurs when the instruction in the APU reports an exception while a second instruction in the conversion unit (CU) is waiting to be handed off to the arithmetic processing unit (APU). Consider the case of two FMUL instructions:

```
FMULX FP0,FP1 (which results in an exception)
FMUL.B <ea>,FP2
```

At the time the second FMUL instruction is initiated, the first FMUL instruction is still executing in the APU. The CU instructs the bus interface unit (BIU) to fetch the program counter, and prefetch the byte operand. Since the CU cannot convert the byte operand, it instructs the BIU to encode a null (CA=1, IA=1) in the response CIR, and waits to hand



instruction to the APU. When the first FMUL instruction finally finishes in the APU, the APU reports an exception, the second FMUL instruction is in the middle of the instruction, hence a take mid-instruction exception is taken. Note that in this case, the destination operand is a floating-point register, and therefore the effective address calculation for the destination operand in the mid-instruction stack frame of the MPU is undefined.

The third point at which the FPCP can indicate an exception to the main processor is in response to a protocol violation. If an unexpected access to a coprocessor interface register causes a protocol violation, the FPCP immediately encodes the response CIR to the take mid-instruction exception primitive with the protocol violation vector number. This allows the protocol violation handler to determine the cause of the violation (either an illegal primitive from the FPCP or an illegal access by the MPU) and perform necessary action. Since an FPCP protocol violation is a catastrophic error, and the FPCP cannot return an illegal primitive, the only appropriate action is to abort the task that detected the protocol violation.

The basic protocol followed in response to a coprocessor-detected exception is:

1. The FPCP encodes the appropriate take exception primitive (pre- or mid-instruction), along with the vector number, in the response CIR.
2. The MPU reads the response CIR (usually in an attempt to initiate the next instruction) and receives the take exception request.
3. The MPU acknowledges the request by writing an exception acknowledge to the control CIR. The appropriate stack frame is then stored in memory, and control is transferred to the exception handler routine.
4. The response to the exception acknowledge differs for the type of exception and for the FPCP, as follows:
 - a. Protocol violation:
 - MC68881 — Aborts all internal operations that may be active and enters the idle state.
 - MC68882 — Same as MC68881.
 - b. BSUN and F-line (detected by the coprocessor):
 - MC68881 — Clears the exception and enters the idle state.
 - MC68882 — Same as MC68881.
 - c. Arithmetic (Operr, Overflow, Underflow, Divide by zero, Inexact result):
 - MC68881 — Clears the exception and enters the idle state.
 - MC68882 — Refer to **5.2.2 Exception Handler Code**.

The following paragraphs discuss the exception vector assignments used by the FPCP, and each of the exception types that can be detected by the FPCP.

The M68000 Family of processors uses a data structure called the exception vector table as a localized dispatching point for all exceptional conditions that may occur in a system. The exception vector table is a 1024-byte structure made up of 256 long word entries. Each entry in the table is a pointer to the routine that services a specific exceptional occurrence. When an exception occurs, the processor supplies an index that selects the vector entry for the exception. The index, called the vector number, is an 8-bit value that is multiplied by four to calculate an offset into the vector table. Of the 256 possible vector numbers, 64 are reserved by Freescale for definition by M68000 Family devices; the remaining 192 are for definition by system designers.

64 reserved vectors, the MPU defines all but 25. The FPCP utilizes three of the same entries defined by the MPU and defines seven additional vectors to support floating-point exceptions. The vectors defined by the FPCP are shown in Table 6-1. The vector number is the value (shown in decimal) that is encoded in the appropriate take exception response primitive (except for the FTRAPcc vector number, which is generated internally by the MPU). The vector offset is the location of the corresponding entry in the vector table. The MPU adds the vector offset to the value contained in the vector base register to calculate the absolute address of the vector. Refer to the appropriate main processor user's manual for further information on the exception processing operations performed by the MPU and for the full definition of the exception vector table.

Table 6-1. MC68881/MC68882 Exception Vector Assignments

Vector Number (Decimal)	Vector Offset (Hexadecimal)	Assignment
7	\$01C	FTRAPcc Instruction
11	\$02C	F-Line Emulator
13	\$034	Coprocessor Protocol Violation
48	\$0C0	Branch or Set on Unordered Condition
49	\$0C4	Inexact Result
50	\$0C8	Floating-Point Divide by Zero
51	\$0CC	Underflow
52	\$0D0	Operand Error
53	\$0D4	Overflow
54	\$0D8	Signaling NAN

The following paragraphs describe the causes for each exception, what information is available to the trap handler, and what results occur if traps are enabled or disabled. FPCP instruction exceptions arise from the detection of abnormal conditions during coprocessor instruction execution. All coprocessor-detected instruction exceptions are enabled or disabled via the FPCR ENABLE byte.

Any of eight exception conditions can be detected during the execution of a floating-point instruction. The location of the exception bits in the EXC and ENABLE bytes (contained in the FPSR and FPCR registers, respectively) is shown in Figure 6-1. If more than one enabled exception occurs during the same instruction, then the highest priority instruction trap is taken (BSUN is the highest; INEX2/INEX1 is the lowest). When multiple exceptions occur, the FPCP traps to the highest priority exception that is enabled, and the lower priority exception does not cause a second trap. It is the programmer's responsibility to determine if any of the exception bits that have lower priority than the exception taken are set.

FPCP instruction exceptions that arise from the move floating-point data register to external destination instructions are reported to the MPU as mid-instruction exceptions. All other MC68881-detected instruction exceptions are reported as pre-instruction exceptions, and all other MC68882-detected instruction exceptions are reported as either pre-instruction or mid-instruction exceptions. The FPCP move multiple and move system control register instructions cannot generate coprocessor detected instruction exceptions. The FSAVE instruction can generate a coprocessor-detected exception only when it interrupts an FSAVE

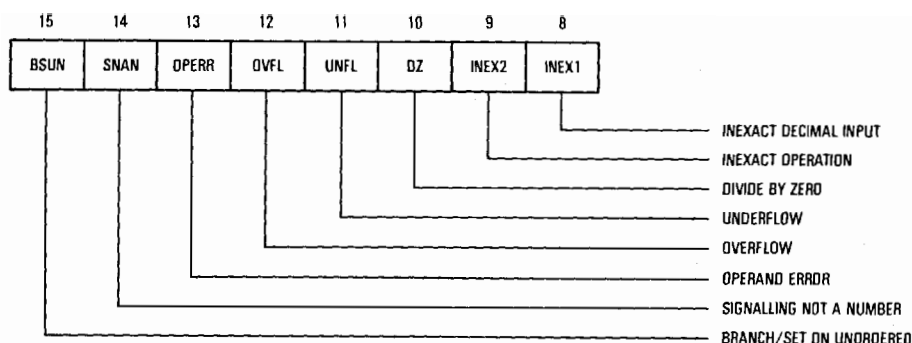


Figure 6-1. EXC and ENABLE Byte Bit Assignments

or FRESTORE operation in progress. The FRESTORE instruction can generate coprocessor-detected instruction exceptions only when the state frame format written to the coprocessor is not recognized.

In the following exception descriptions, the term "intermediate result" is used frequently. During the execution of a floating-point operation, the FPCP arithmetic processing unit (APU) contains a 67-bit mantissa (for rounding purposes) and a 17-bit exponent (to ensure that overflow or underflow can never occur during the main algorithm). At the end of the operation, this intermediate result must be stored in a floating-point data register, in an MPU data register, or in memory. This intermediate result is checked for underflow, rounded, and checked for overflow to obtain the final result.

6.1.1 Branch/Set on Unordered (BSUN)

The BSUN exception is the result of performing a conditional test associated with the FBcc, FDBcc, and FTRAPcc instructions when an unordered condition is present. (An unordered condition occurs when an input to an arithmetic operation is a NAN.) The BSUN exception can only occur during FPCP conditional instructions with the following IEEE nonaware branch condition predicates:

GT	Greater Than	GL	Greater Than or Less Than
NGT	Not Greater Than	NGL	Not Greater Than or Less Than
GE	Greater Than or Equal	GLE	Greater Than or Less Than or Equal
NGE	Not Greater Than or Equal	NGLE	Not Greater Than or Equal Less Than or Equal
LT	Less Than	SF	Signaling False
NLT	Not Less Than	ST	Signaling True
LE	Less Than or Equal	SEQ	Signaling Equal
NLE	Not Less Than or Equal	SNE	Signaling Not Equal

If the APU is busy (MC68881) or if the CU (MC68882) is busy, a null (CA = 1, IA = 1) primitive is returned, and the MPU continues to reexamine the response CIR. If an exception is pending, a take pre-instruction exception primitive is returned. After the appropriate exception handler is executed, the conditional instruction is restarted. When either the APU is idle (MC68881) or the APU and CU are idle (MC68882) and no exceptions are pending, FPCP checks for a BSUN exception, evaluates the conditional predicate, and reports the result to the MPU.

MPU can write to the condition CIR of the MC68882 when both the CU and APU are . If exceptions are enabled and if each of the instructions report an exception, the MC68882 reports the exceptions and executes the handlers, one at a time. The MC68882 restarts the conditional instruction after returning from each exception handler; that is, the MC68882 restarts the instruction twice. It is important to note that the coprocessor completes all previous instructions and the MPU completes any executing exception handler before the conditional instruction checks for a BSUN exception, evaluates the conditional predicate, and reports the result to the MPU.

The FPCP detects a BSUN exception if the conditional predicate is one of the IEEE nonaware branches, and the NAN condition code bit is set. When the FPCP detects this exception, it sets the BSUN bit in the FPSR exception status byte.

Trap Disabled Results: The FPCP evaluates the condition and reports the result to the MPU in the response CIR.

Trap Enabled Results: The FPCP reports a pre-instruction exception to the MPU with the BSUN vector number instead of a true or false indication.

6

The BSUN exception is unique in that the trap is taken before the conditional predicate is evaluated. Furthermore, the instruction that caused the BSUN exception is re-executed following return from the BSUN trap handler. Therefore, it is the responsibility of the trap handler to prevent the conditional instruction from taking the BSUN trap again. Four ways are available to prevent taking the trap again.

The first way involves incrementing the stored program counter in the stack to bypass the conditional instruction. This technique applies to situations where a fall-through is desired. Be aware that accurate calculation of the program counter increment requires detailed knowledge of the size of the conditional instruction being bypassed.

The second method is to clear the NAN bit of the FPSR condition code byte. However, this alone cannot deterministically control the result indication (true or false) which would be returned when the conditional instruction re-executes.

The third method is to disable the BSUN trap. Like the second method, this method cannot control the result indication (true or false) which would be returned when the conditional instruction re-executes.

The fourth method involves examining the condition predicate and setting the condition code in the FPSR accordingly. This technique gives the most control since it is possible to pre-determine the direction of program flow. Bit 7 of the F-line operation word indicates where the conditional predicate is located. If bit 7 is set, the conditional predicate is the lower six bits of the F-line operation word. Otherwise, the conditional predicate is the lower six bits of the instruction word, which immediately follows the F-line operation word. Using the conditional predicate and the table in **4.4.1 IEEE NonAware Tests**, the condition codes can be set to return a known result indication when the conditional instruction is re-executed.

6.1.2 Signaling Not-a-Number

An SNAN is used as an escape mechanism for a user defined, non-IEEE data type. The FPCP never creates an SNAN as a result of an operation; a NAN created by an operand error exception is always a nonsignaling NAN.



an SNAN is an operand involved in an arithmetic instruction, the SNAN bit is set in the SR exception byte. Since the FMOVEM, FMOVE FPCR, and FSAVE instructions do not modify the status bits, they cannot generate exceptions. Therefore, these instructions are useful for manipulating SNANs.

Trap Disabled Results: If the destination data format is S, D, X, or P, then the SNAN bit in the NAN is set to one and the resulting nonsignaling NAN is transferred to the destination. No bits other than the SNAN bit of the NAN are modified, although the input NAN is truncated if necessary. If the destination data format is B, W, or L, then the 8, 16, or 32 most significant bits of the SNAN significand, with the SNAN bit set, are written to the destination.

Trap Enabled Results: For memory or MPU data register destinations, the result is written in the same manner as if the trap were disabled, and then a mid-instruction exception is signaled. If desired, the trap handler can overwrite the result.

For floating-point data register destinations, instruction execution is terminated, and the floating-point data registers are not modified. In this case, the SNAN trap handler should supply the result.

Note that the trap handler should use only the FMOVEM instruction to read or write the floating-point data registers, since FMOVEM cannot generate further exceptions. Also, only an FMOVEM instruction can write a SNAN into a floating-point data register.

6.1.3 Operand Error

The operand error category encompasses problems arising in a variety of operations, and includes those errors not frequent or important enough to merit a specific exception condition. Basically, an operand error occurs when an operation has no mathematical interpretation for the given operands. The possible operand errors are listed in Table 6-2. When an operand error occurs, the OPERR bit is set in the FPSR exception status byte.

Trap Disabled Results: For a memory or MPU data register destination, several possible results can be supplied, depending on the destination size and error type. (An operand error is never generated when the destination is an MPU data register or memory and the destination format is S, D, or X.)

If the operand error is caused by an integer overflow or if the floating-point data register to be stored contains infinity, the result is the largest positive or negative integer that can fit in the specified destination format size. If the destination is B, W, or L and the floating-point number to be stored is a NAN, then the 8, 16, or 32 most significant bits of the NAN significand are stored as the result.

For packed decimal results, if the k factor is greater than +17, the result returned is a packed decimal string that assumes a k factor equal to +17. For packed decimal results where the absolute value of the exponent is greater than 999, the decimal string is returned with the three least significant exponent digits in EXP2, EXP1, and EXP0. The fourth digit, EXP3, is supplied in the most significant four bits of the third byte in the string. Refer to **3.6 DATA FORMAT DETAILS** for the packed decimal string format.

If the destination is a floating-point data register, an extended precision nonsignaling NAN (with all ones mantissa) is stored in the destination floating-point data register.

Table 6-2. Possible Operand Errors

Instruction	Condition Causing Operand Error
FACOS	Source is $\pm$ infinity, $> +1$, or < -1
FADD	$(+infinity) + (-infinity)$ or $(-infinity) + (+infinity)$
FASIN	Source is $\pm$ infinity, $> +1$, or < -1
FATANH	Source is $> +1$, or < -1 , Source = $\pm$ infinity
FCOS	Source is $\pm$ infinity
FDIV	0/0 or infinity/infinity
FGETEXP	Source is $\pm$ infinity
FGETMAN	Source is $\pm$ infinity
FLOG10	Source is < 0 , Source = $-infinity$
FLOG2	Source is < 0 , Source = $-infinity$
FLOGN	Source is < 0 , Source = $-infinity$
FLOGNP1	Source is < -1 , Source = $-infinity$
FMOD	Floating-Point Data Register is $\pm$ infinity or Source is 0, Other Operand is Not a NAN
FMOVE to B, W, or L	Integer Overflow/Underflow, Source is Non-Signaling NAN, or Source is $\pm$ infinity
FMOVE to P	Result Exponent > 999 (Decimal) or k-Factor $> +17$
FMUL	One Operand is 0, Other Operand is $\pm$ infinity
FREM	Floating-Point Data Register is $\pm$ infinity or Source is 0, Other Operand is Not a NAN
FSCALE	Source is $\pm$ infinity, Other Operand is Not a NAN
FSGLDIV	0/0 or infinity/infinity
FSGLMUL	One operand is 0, Other Operand is infinity
FSIN	Source is $\pm$ infinity
FSINCOS	Source is $\pm$ infinity
FSQRT	Source < 0 , Source = $-infinity$
FSUB	Source and floating-point data register are $+infinity$ or source and FpN are $-infinity$
FTAN	Source is $\pm$ infinity

Trap Enabled Results: For memory or MPU data register destinations, the destination operand is written as if the trap were disabled, and then a take exception primitive is returned to the MPU. This can only occur for the FMOVE FpM, <ea> instruction, and the exception is reported as a mid-instruction exception. If desired, the trap handler can overwrite the result generated by the FPCP.

If the destination is a floating-point data register, the register is not modified by the FPCP. In this case, the trap handler should generate the appropriate result.

To enable the trap handler to return a result for memory or MPU data register destinations, the MPU and the FPCP supply:

1. The address of the instruction where the error occurred (in the FPIAR). By examining the instruction, the trap handler may determine the operation being performed, the value of the second operand (for dyadic instructions), and the destination location.
2. The address of the destination in the mid-instruction stack frame (at offset +\$10). This allows the trap handler to overwrite the NAN, if necessary, without recalculating the effective address.



able the trap handler to return a result for floating-point data register destinations, PU and the FPCP supply:

1. The address of the instruction where the error occurred (in the FPIAR). By examining the instruction, the trap handler may determine the operation being performed, the value of the second operand (for dyadic instructions), and the destination location.
2. The exceptional operand in the FPCP idle state frame. For additional FSAVE state frame information, refer to **6.4.2 State Frames**. When an SNAN trap occurs, the exceptional operand is the source input argument converted to extended precision.

Note that the trap handler should use only the FMOVEM instruction to read or write the floating-point data registers since FMOVEM cannot generate further exceptions or change the condition codes.

6.1.4 Overflow

An overflow occurs when the intermediate result of an arithmetic operation is too large to be represented in a floating-point data register using the selected rounding precision. A store-to-memory operation overflows when the value in the source floating-point data register is too large to be represented in the destination format.

Overflow is detected for arithmetic operations where the destination is a floating-point data register when the intermediate result exponent is greater than or equal to the maximum exponent value of the selected rounding precision. (Refer to **2.2.2 FPCR Mode Control Byte**.) Overflow is detected for store-to-memory operations when the intermediate result exponent is greater than or equal to the maximum exponent value of the destination data format. Overflow can only occur when the destination is in the S, D, or X format. Overflows when converting to the B, W, or L integer and packed decimal formats are included as operand errors. Refer to **3.6 DATA FORMAT DETAILS** for the maximum exponent value for each format. At the end of any operation that could potentially overflow, the intermediate result is checked for underflow, rounded, and checked for overflow before it is stored to the destination. If overflow occurs, the OVFL bit is set in the FPSR exception byte.

NOTE

An overflow can occur when the destination is a floating-point data register and the selected rounding precision is single or double even if the intermediate result is small enough to be represented as an extended precision number. The intermediate result is rounded to the selected precision (both the mantissa and the exponent), and then the rounded result is stored in extended precision format. If the magnitude of the intermediate result exceeds the range of the selected rounding precision format, an overflow occurs. The FSGLMUL and FSGLDIV instructions are the exceptions in that, although the mantissa of the intermediate result is rounded to single precision, the exponent remains an extended format exponent. Therefore, those instructions can never report an overflow as long as the intermediate result is small enough to be represented in extended precision format.

Trap Disabled Results: The current rounding mode determines the value to be stored at the destination, as follows:

RN	Infinity, with the sign of the intermediate result
RZ	Largest magnitude number, with the sign of the intermediate result
RM	For positive overflow, largest positive number For negative overflow, $-\infty$
RP	For positive overflow, $+\infty$ For negative overflow, largest negative number

Trap Enabled Results: The result stored in the destination is the same as the result stored when the trap is disabled, and a take exception primitive is returned to the MPU. If the destination is memory or an MPU data register, the operand is stored, and then a take mid-instruction exception primitive is issued. If the destination is a floating-point data register, a take exception primitive is returned when the MPU reads the response CIR of the FPCP. Since the MC68881 does not allow multiple floating-point concurrency, a take pre-instruction exception is reported when the MPU attempts the next floating-point instruction. The MC68882 can report an exception as a mid-instruction exception on a subsequent floating-point instruction.

6

The address of the instruction that causes the overflow is available to the trap handler in the FPIAR. By examining the instruction, the trap handler can determine the arithmetic operation type and destination location. The trap handler can execute an FSAVE instruction to obtain additional information. When an FSAVE is executed, the exceptional operand is stored in the state frame. Refer to **6.4.2 State Frames** for details of the FSAVE instruction state frames. When an overflow occurs, the exceptional operand is defined differently for various destination types:

1. Memory or MPU data register destination — the value in the exceptional operand is the intermediate result mantissa rounded to the destination precision, with a 15-bit exponent biased as a normal extended precision number. In the case of a memory destination, the evaluated effective address of the operand is available in the MPU mid-instruction stack frame (at offset +\$10). This allows the trap handler to overwrite the default result, if necessary, without recalculating the effective address.
2. Floating-point data register destination — the value in the exceptional operand is the intermediate result rounded to extended precision, with an exponent bias of \$3FFF-\$6000 rather than \$3FFF. The additional bias of -\$6000 is used to “wrap” the 17-bit intermediate value into a value that can be represented in 15 bits. To recover the 17-bit twos-complement exponent of the intermediate result, the 15-bit exponent of the exceptional operand should be sign extended to at least 17 bits (i.e., if it is manipulated in an MPU data register, it is sign extended to a long word value), and then the bias of \$3FFF-\$6000 should be subtracted from that number. Note that for most operations, the intermediate exponent value does not exceed 32,767 and thus can be contained in a 16-bit integer. However, a completely general exception handler should calculate a 17-bit exponent value.

In addition to normal overflow, the exponential instructions implemented by the FPCP (eX, 10X, 2X, SINH, COSH, and FSCALE) may generate results that overflow the 17-bit exponent used for intermediate results. For example, the eX function can easily overflow the 17-bit intermediate exponent if the source value is a large number ($x \geq +18,192$). When such an overflow occurs (called a catastrophic overflow), the exceptional operand exponent value is set to \$0000. This value is easily distinguished from the exceptional operand exponent values produced by normal overflow processing. The



smallest exceptional operand exponent value that can be produced by a normal overflow is \$1FFF (\$04000 + \$3FFF - \$6000, truncated to 15 bits), while the largest exceptional operand exponent value is \$7FFF (\$0A000 + \$3FFF - \$6000, truncated to 15 bits). The catastrophic overflow exceptional operand exponent value of \$0000 is produced any time the unbiased exponent of the calculated intermediate result is a value greater than \$0A000.

Note that the trap handler should use only the FMOVEM instructions to read or write the floating-point data registers since FMOVEM cannot generate further exceptions or change the condition codes.

6.1.5 Underflow

An underflow occurs when the intermediate result of an arithmetic operation is too small to be represented as a normalized number in a floating-point data register using the selected rounding precision. A store-to-memory operation underflows when the value in the source floating-point data register is too small to be represented in the destination format as a normalized number. Underflow is detected for arithmetic operations where the destination is a floating-point data register when the intermediate result exponent is less than or equal to the minimum exponent value of the selected rounding precision (refer to 2.2.2 FPCR Mode Control Byte).

Underflow is detected for store-to-memory operations when the intermediate result exponent is less than or equal* to the minimum exponent value of the destination data format.

Underflow can only occur when the destination format is S, D, or X. When the destination format is packed decimal, underflows are included as operand errors. When the destination format is B, W, or L, the conversion underflows to zero without causing either an underflow or an operand error. See 3.6 DATA FORMAT DETAILS for the minimum exponent value for each format.

At the end of any operation that could potentially underflow, the intermediate result is checked for underflow, rounded, and checked for overflow before it is stored at the destination. If an underflow occurs, the UNFL bit is set in the FPSR exception status byte.

NOTE

An underflow can occur when the destination is a floating-point data register and the selected rounding precision is single or double even if the intermediate result is large enough to be represented as an extended precision number. The intermediate result is rounded to the selected precision (both the mantissa and the exponent), and then the rounded result is stored in extended precision format. If the magnitude of the intermediate result is too small to be represented in the selected rounding precision format, an underflow occurs. The FSGLMUL and FSGLDIV are exceptions in that, although the mantissa of the intermediate result is rounded to single precision, the exponent remains an extended precision format

*Underflow is NOT detected for intermediate result exponents that are equal to the extended precision minimum exponent, since the explicit integer part bit of extended precision permits representation of normalized numbers with a minimum extended precision exponent.

exponent. Therefore, these instructions can never report an underflow as long as the intermediate result is large enough to be represented in the extended precision format.

Trap Disabled Results: The result that is stored in the destination is either a denormalized number or zero. The intermediate result is always normalized because the FPCP ALU and temporary registers use a 17-bit exponent. Denormalization is accomplished by shifting the mantissa of the intermediate result to the right while incrementing the exponent until it is equal to the denormalized exponent value for the destination format. The denormalized intermediate result is rounded to the selected rounding precision or destination format.

If, in the process of denormalizing the intermediate result, all of the significant bits are shifted off to the right, the selected rounding mode determines the value to be stored at the destination, as follows:

Rounding Mode	Result
RN	Zero, with the sign of the intermediate result
RZ	Zero, with the sign of the intermediate result
RM	For positive underflow, + zero For negative underflow, smallest denormalized negative number
RP	For positive underflow, smallest denormalized positive number For negative underflow, – zero

Trap Enabled Results: The result stored in the destination is the same as the result stored when traps are disabled, and a take exception primitive is returned to the MPU. If the destination is memory or an MPU data register, the operand is stored, and then a take mid-instruction exception primitive is issued. If the destination is a floating-point data register, a take exception primitive is returned when the MPU reads the response CIR of the FPCP. Since the MC68881 does not allow multiple floating-point concurrency, the exception is always reported as a pre-instruction exception when the next floating-point instruction is attempted. The MC68882, however, may report an exception as a mid-instruction exception on a subsequent floating-point instruction.

The address of the instruction that caused the underflow is available to the trap handler in the FPIAR. By examining the instruction, the trap handler can determine the arithmetic operation type and destination location. The trap handler can execute an FSAVE instruction to obtain additional information. When an FSAVE instruction is executed, the exceptional operand is stored in the state frame. Refer to **6.4.2 State Frames** for details of FSAVE state frames. The exceptional operand is defined differently for various destination types:

1. Memory or MPU data register destination — the value in the exceptional operand is the intermediate result mantissa rounded to the destination precision, with a 15-bit exponent biased as a normal extended precision number. In the case of a memory destination, the evaluated effective address of the operand is available in the MPU mid-instruction stack frame (at offset +\$10). This allows the trap handler to overwrite the default result, if necessary, without recalculating the effective address.
2. Floating-point data register destination — the value in the exceptional operand is the intermediate result mantissa rounded to extended precision, with an exponent bias of \$3FFF+\$6000 rather than \$3FFF. The additional bias of +\$6000 is used to “wrap” the 17-bit intermediate value into a value that can be represented in 15 bits. To recover the 17-bit two’s-complement exponent of the intermediate result, the 15-bit exponent

if the exceptional operand is sign extended to at least 17 bits (i.e., if it is manipulated in an MPU data register, it is sign extended to a long-word value), and then the bias of $\$3FFF + \6000 is subtracted from that number. Note that for most operations, the intermediate exponent value is not less than $-32,768$, and thus can be contained in a 16-bit integer. However, a completely general exception handler should calculate a 17-bit exponent value.

In addition to normal underflow, the exponential instructions implemented by the FPCP (e_x , 10_x , 2_x , $SINH$, $COSH$, and $FSCALE$) may generate results that underflow the 17-bit exponent used for intermediate results. For example, the e_x function can easily underflow the 17-bit intermediate exponent if the source value is a large number ($x \leq -8,192$). When such an underflow occurs (called a catastrophic underflow), the exceptional operand exponent value is set to $\$0000$. This is the smallest exception operand exponent value that can be produced by a normal underflow ($\$16001 + \$3FFF + \$6000$, truncated to 15 bits), while the largest underflow exponent value is $\$5FFF$ ($\$1C000 + \$3FFF + \$6000$, truncated to 15 bits). The catastrophic underflow exceptional operand exponent value of $\$0000$ is produced any time the unbiased 17-bit exponent of a calculated intermediate result has a value less than or equal to $\$16001$.

Note that the trap handler should use only the FMOVEM instructions to read or write to the floating-point data registers since FMOVEM cannot generate further exceptions or change the condition codes.

NOTE

The IEEE standard defines two causes of an underflow:

1. When a result is very small, the absolute value of the number is less than the minimum number that can be represented by a normalized number in a specific format.
2. When loss of accuracy occurs while attempting to calculate a very small number (a loss of accuracy also causes an inexact exception).

The IEEE standard specifies that if the underflow trap is disabled, an underflow should only be signaled when both of these cases are satisfied (i.e., the result is too small to represent with a given format, and there is a loss of accuracy during the calculation of the final result). If the trap is enabled, the underflow should be signaled any time a tiny result is produced, regardless of whether accuracy is lost in calculating it.

The FPCP UNFL bit in the AEXC byte of the FPSR implements the IEEE trap disabled definition, since it is only set when a very small number is generated and accuracy has been lost when calculating that number. The UNFL bit in the EXC byte implements the IEEE trap enabled definition, since it is set anytime a tiny number is generated. Thus, if the FPCP underflow trap is enabled, a trap occurs when very small size alone is detected (as the IEEE standard specifies) to support the emulation of machines that underflow to zero, rather than using the IEEE gradual underflow method (i.e., denormalized numbers). If the underflow trap is disabled, the UNFL bit in the AEXC byte may be examined at the end of a calculation to determine if any result produced during the operation required representation as a denormalized number, and accuracy was lost when denormalizing and rounding that result.

This exception occurs when a zero divisor occurs in a division, or when a transcendental function is asymptotic with infinity as the asymptote. Table 6-3 lists the instructions that can generate the divide-by-zero exception. When a divide-by-zero is detected, the DZ bit is set in the FPSR exception status byte.

Table 6-3. Possible Divide-by-Zero Exceptions

Instruction	Input Operand Value
FATANH	Source Operand = ± 1
FDIV	Source Operand = 0 and FPn is Not a NAN, Infinity, or Zero
FLOG10	Source Operand = 0
FLOG2	Source Operand = 0
FLOGN	Source Operand = 0
FLOGNP1	Source Operand = -1
FSGLDIV	Source Operand = 0 and FPn is Not a NAN, Infinity, or Zero

6

Trap Disabled Results: Store the following results in the destination floating-point data register:

- For the FDIV and FSGLDIV instructions, return an infinity with the sign set to the exclusive OR of the signs of the input operands.
- For the FLOGx instructions, return minus infinity.
- For the FATANH instruction, return a +infinity if the source operand is -1 ; or a $-$ infinity if the source operand is $+1$.

Trap Enabled Results: The destination floating-point data register is not modified, and a take exception primitive is returned when the MPU reads the response CIR of the FPCP. Since the MC68881 does not allow multiple floating-point concurrency, the exception is always reported as a pre-instruction exception when the next floating-point instruction is attempted. The MC68882, however, may report an exception as a mid-instruction exception on a subsequent floating-point instruction. The trap handler must generate a result to store in the destination.

To assist the trap handler in this function, the FPCP supplies:

1. The address of the instruction where the divide-by-zero occurred (in the FPIAR). By examining this instruction, the trap handler can determine the operation being performed, the value of the source operand (for dyadic instructions), and the destination floating-point register number.
2. The FSAVE instruction that places the exceptional operand in a state frame. For additional FSAVE state frame information, refer to **6.4.2 State Frames**. The exceptional operand is the source input argument converted to extended precision.

Note that the trap handler should use only the FMOVEM instruction to read or write the floating-point data registers, since FMOVEM cannot generate further exceptions or change the condition codes.

The FPCP provides two inexact bits (INEX1 and INEX2) to help distinguish between inexact results generated by decimal input (INEX1) and other inexact results (INEX2). Two inexact bits are useful in instructions in which both types of inexact results can occur, such as:

FDIV.P #7E – 1,FP3

In this case, the packed decimal to extended precision conversion of the immediate source operand causes an inexact error to occur which is signaled as INEX1. Furthermore, the subsequent divide might also produce an inexact result and cause INEX2 to be set. Therefore, the FPCP provides two inexact bits in the FPSR exception status byte to distinguish these two cases.

Note that only one inexact exception vector number is generated by the FPCP. If either of the two inexact exceptions is enabled, the MPU fetches the inexact exception vector, and the exception handler routine is initiated. Refer to **6.1.8 Inexact Result on Decimal Input** for a discussion of INEX1.

In a general sense, INEX2 is the condition that exists when any operation, except the input of a packed decimal number, creates a floating-point intermediate result whose infinitely precise mantissa has too many significant bits to be represented exactly in the selected rounding precision (refer to **2.2.2 FPCR Mode Control Byte**) or in the destination data format. If this condition occurs, the INEX2 bit is set in the FPSR exception status byte, and the infinitely precise result is rounded as described in the next paragraph.

The FPCP supports the four rounding modes specified by the IEEE standard. These modes are round to nearest (RN), round toward zero (RZ), round toward plus infinity (RP), and round toward minus infinity (RM). The rounding definitions are:

Rounding Mode	Result
RN	The representable value nearest to the infinitely precise intermediate value is the result. If the two nearest representable values are equally near (a tie), then the one with the least significant bit equal to zero (even) is the result. This is sometimes referred to as “round nearest, even”.
RZ	The result is the value closest to, and no greater in magnitude than, the infinitely precise intermediate result. This is sometimes referred to as the “chop mode”, since the effect is to clear the bits to the right of the rounding point.
RM	The result is the value closest to and no greater than the infinitely precise intermediate result (possibly minus infinity).
RP	The result is the value closest to and no less than the infinitely precise intermediate result (possibly plus infinity).

The RM and RP rounding modes are often referred to as “directed rounding modes” and are useful in interval arithmetic. Rounding is accomplished using the intermediate result format shown in Figure 6-2.

Depending on the selected rounding precision or destination data format in effect, the location of the least significant bit of the fraction and the locations of the guard, round, and sticky bits in the 67-bit intermediate result mantissa varies.

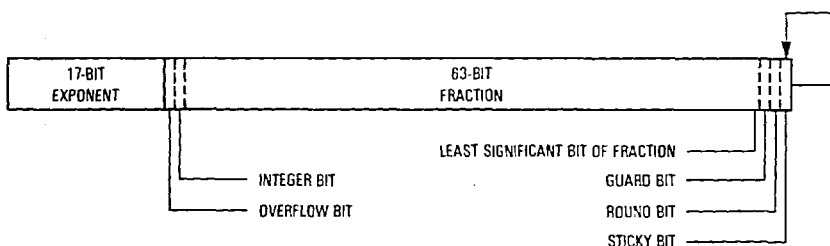


Figure 6-2. Intermediate Result Format

6

The guard and round bits are always calculated exactly. The sticky bit is used to create the illusion of an infinitely wide intermediate result mantissa. As shown by the arrow in Figure 6-2, the sticky bit is the logical OR of all the bits in the infinitely precise result to the right of the round bit. During the calculation stage of an arithmetic operation, any nonzero bits generated that are to the right of the round bit set the sticky bit (which is used in rounding) to one. Because of the sticky bit, the rounded intermediate result for all required IEEE arithmetic operations in the round-to-nearest mode is in error by no more than one-half unit in the last place. For transcendental instructions, the result may not be this accurate (refer to **4.3 COMPUTATIONAL ACCURACY**).

NOTE

When the FPCP is programmed to operate in the single or double precision rounding mode, a method referred to as “range control” is used to assure correct emulation of a machine that only supports single or double precision arithmetic. When the FPCP performs any calculation, the intermediate result is in the format shown in Figure 6-2, and a rounded result stored into a floating-point data register is always in the extended precision format. However, if the single or double precision rounding mode is in effect, the final result generated by the FPCP is within the range of the format (except for the FSGLDIV and FSGLMUL instructions, as described in **4.5.5.2 UNDERFLOW, ROUND, OVERFLOW**).

Range control is accomplished by not only rounding the intermediate result mantissa to the specified precision, but also checking the 17-bit intermediate exponent to ensure that it is within the representable range of the selected rounding precision format. If the intermediate exponent exceeds the range of the selected precision, the exponent value appropriate for an underflow or overflow is stored as the result in the 15-bit extended precision format exponent. For example, if the rounding precision and mode is single/RM and the result of an arithmetic operation overflows the magnitude of the single precision format, the largest normalized single precision value is stored as an extended precision number in the destination floating-point data register (i.e., an unbiased 15-bit exponent of \$00FF and a mantissa of \$FFFFFF0000000000). If an infinity is the appropriate result for an underflow or overflow, the infinity value for the destination data type is stored as the result (i.e., an exponent with the maximum value and a mantissa of zero).

Figure 6-3 shows the algorithm that is used to round an intermediate result to the selected rounding precision or destination data format. If the destination is a floating-point register,

```

BEGIN
  IF GUARD, ROUND AND STICKY=0
    THEN (RESULT IS EXACT)
      DON'T SET INEX2
      DON'T CHANGE THE INTERMEDIATE RESULT
    ELSE (RESULT IS INEXACT)
      SET INEX2 IN THE FPSR EXC BYTE
      SELECT THE ROUNDING MODE
        RM: IF INTERMEDIATE RESULT IS NEGATIVE
          THEN ADD 1 TO LSB
        RN: IF GUARD=1 AND ROUND AND STICK=0 (TIE CASE)
          THEN IF LSB=1 ADD 1 TO LSB
          ELSE IF GUARD=1 ADD 1 TO LSB
          ENDIF
        RP: IF INTERMEDIATE RESULT IS POSITIVE
          THEN ADD 1 TO LSB
        RZ: (FALL THROUGH; GUARD, ROUND AND STICKY ARE CHOPPED)
      END SELECT
      IF OVERFLOW=1
        THEN
          SHIFT MANTISSA RIGHT BY ONE BIT
          ADD 1 TO THE EXPONENT
        END IF
      SET GUARD, ROUND AND STICK TO 0
    END IF
  END
END

```

Figure 6-3. Rounding Algorithm

the rounding boundary is determined by the selected rounding precision in the FPSR. If the destination is external memory or an MPU data register, the rounding boundary is determined by the destination data format. If the rounded result of an operation is not exact, then the INEX2 bit is set in the FPSR exception status byte.

Trap Disabled Results: The rounded result is delivered to the destination.

Trap Enabled Results: The rounded result is delivered to the destination, and an exception is reported to the MPU. If the destination is memory or an MPU data register, a take mid-instruction exception primitive is returned immediately after the operand is stored. If the destination is a floating-point data register, a take exception primitive is returned when the MPU reads the response CIR of the FPCP. Since the MC68881 does not allow multiple floating-point concurrency, the exception is always reported as a pre-instruction exception when the next floating-point instruction is attempted. The MC68882, however, may report an exception as a mid-instruction exception on a subsequent floating-point instruction.

The address of the instruction that generated the inexact result is available to the trap handler in the FPIAR. The trap handler can determine the location of the operand(s) by examining the instruction. In the case of a memory destination, the evaluated effective address of the operand is available in the MPU mid-instruction stack frame (at offset +\$10). When an FSAVE is executed by an inexact trap handler, the value of the exceptional operand in the state frame is not defined (refer to **6.4.2 State Frame**). An inexact exception differs from the other exceptions in this respect. If an inexact condition is the only exception that occurred during the execution of an instruction, the value of the exceptional operand is invalid. If multiple exceptions occur during an instruction, the exceptional operand value is related to a higher priority exception.



that the trap handler should use only the FMOVEM instruction to read or write the floating-point data registers, since FMOVEM cannot generate further exceptions or change the condition codes.

NOTE

The IEEE standard specifies that inexactness should be signaled on overflow as well as for rounding. The FPCP implements this via the INEX bit in the FPSR AEXC byte. However, the standard also indicates that the inexact trap should be taken if an overflow occurs with the overflow trap disabled and the inexact trap enabled. Therefore, the FPCP takes the inexact trap if this combination of conditions occurs, even though the INEX1 or INEX2 bits may not be set in the FPSR EXC byte. In this case, INEX is set in the AEXC byte and OVFL is set in both the EXC and AEXC bytes.

6

6.1.8 Inexact Result on Decimal Input

In a general sense, inexact result 1 (INEX1) is the condition that exists when a packed decimal operand cannot be converted exactly to extended precision in the current rounding mode. If this condition occurs, the INEX1 bit is set in the FPSR exception status byte, and the result of the decimal-to-binary conversion is rounded to extended precision (regardless of FPSR mode byte rounding precision) as shown in Figure 6-3. The FPCP provides two inexact bits (INEX1 and INEX2) to help distinguish between inexact results generated by decimal input conversions (INEX1) and other inexact results (INEX2).

Trap Disabled Results: If the instruction is an FMOVE to a floating-point data register, the rounded result is stored in the floating-point data register. If the instruction is not an FMOVE, the rounded result is used in the calculation.

Trap Enabled Results: The result is generated in the same manner as if traps were disabled, except that a take exception primitive is returned when the MPU reads the response CIR of the FPCP. Since the MC68881 does not allow multiple floating-point concurrency, the exception is always reported as a pre-instruction exception when the next floating-point instruction is attempted. The MC68882, however, may report an exception as a mid-instruction exception on a subsequent floating-point instruction.

The address of the instruction that caused the inexact decimal conversion is available to the trap handler in the FPIAR. The trap handler can determine the location of the decimal string by examining the instruction, although the effective address of the string must be recalculated (if possible) by the trap handler. When an FSAVE is executed by an inexact trap handler, the value of the exceptional operand in the state frame is not defined (refer to **6.4.2 State Frame**). An inexact exception differs from the other exceptions in this respect. If the inexact conversion is the only exception that occurs during the execution of an instruction, the value of the exceptional operand is invalid. If multiple exceptions occur during an instruction, the exceptional operand value is related to a higher priority exception.

Note that the trap handler should use only the FMOVEM instruction to read or write the floating-point data registers, since FMOVEM cannot generate further exceptions or change the condition codes.



... and triple instruction exceptions may be generated by a single instruction in a few cases. When multiple exceptions occur with traps enabled for more than one exception class, only the highest priority exception trap is taken; the other enabled exceptions do not cause a trap. The higher priority trap handler must check for multiple exceptions. The priority of the traps is as follows:

BSUN	Highest Priority
SNAN	
OPERR	
OVFL	
UNFL	
DZ	
INEX2/INEX1	Lowest Priority

The multiple instruction exceptions that can occur are:

- SNAN and INEX1
- OPERR and INEX2
- OPERR and INEX1
- OVFL and INEX2 and/or INEX1
- UNFL and INEX2 and/or INEX1
- INEX2 and INEX1

6.1.10 IEEE Exception and Trap Compatibility

The IEEE standard defines only five exceptions. The FPCP FPSR AEXC byte contains bits representing these five exceptions, which are defined to function exactly as the standard specifies the exceptions. However, it may be more useful to differentiate the IEEE required exceptions into the eight exceptions represented in the FPSR EXC byte. Since the FPCP uses the bits in the FPSR EXC byte and the FPCR ENABLE byte to determine when to trap, there are seven possible instruction traps defined (INEX1 and INEX2 share one exception vector) instead of the five defined by the standard.

If it is necessary to write an application program that only supports the five IEEE specified traps, the BSUN, SNAN, and OPERR exception vectors should be set to point to the same handler routine. This allows the FPCP to support the invalid operation exception defined in the IEEE standard, which is represented by the invalid operation (IOP) bit in the AEXC byte.

To satisfy other requirements in the IEEE standard, the FPCP does the following:

1. A one is ORed into the AEXC byte IOP bit if the BSUN, SNAN, or OPERR bit is set in the EXC byte.
2. A one is ORed into the AEXC byte UNFL bit only if both the UNFL and the INEX2 bits of the EXC byte are set. However, per the IEEE standard, the underflow trap is based only on the UNFL bit in the EXC byte.
3. A one is ORed into the AEXC byte INEX bit if the INEX1, INEX2 or OVFL bit is set in the EXC byte.
4. The IEEE standard requires that an inexact trap be taken if it is enabled, an overflow occurs, and the overflow trap is disabled. Thus, if the OVFL bit is set in the EXC byte, the OVFL bit is not set in the ENABLE byte, and the INEX2 bit is set in the ENABLE byte, then the inexact trap is taken.



Equations for items 1, 2, and 3 are:

$$\begin{aligned} \text{XC(IOP)} &= \text{AEXC(IOP)} \vee \text{EXC(BSUN} \vee \text{SNAN} \vee \text{OPERR)} \\ \text{AEXC(UNFL)} &= \text{AEXC(UNFL)} \vee \text{EXC(UNFL} \wedge \text{INEX2)} \\ \text{AEXC(INEX)} &= \text{AEXC(INEX)} \vee \text{EXC(INEX1} \vee \text{INEX2} \vee \text{OVFL)} \end{aligned}$$

The equation for item 4 (inexact trap taken) is:

$$\text{Inexact Trap} = \text{[EXC(OVFL)} \vee \text{EXC(INEX2)]} \wedge \text{ENABLE(INEX2)} \vee \text{[EXC(INEX1)} \wedge \text{ENABLE(INEX1)]}$$

where:

“v” = logical OR

“^” = logical AND

6.1.11 Illegal Command Words

Illegal coprocessor commands are coprocessor command word bit patterns that are not implemented by the FPCP. The FPCP reports illegal coprocessor commands as pre-instruction exceptions, using the F-line emulator vector number. The specific illegal command word bit patterns are defined in **4.7 INSTRUCTION ENCODING DETAILS**.

FPCP instructions consist of an operation word, a coprocessor command word (if any), and extension words (if any). The MPU detects an illegal operation word and the FPCP detects an illegal command word.

For the case where a coprocessor-detected instruction trap is pending when the MPU writes an illegal coprocessor command to the FPCP command CIR, the coprocessor first reports the pending instruction exception as a pre-instruction exception. Following exception processing of the instruction exception, the MPU resumes execution of the main program at the beginning of the illegal coprocessor command, by writing to the command CIR again. The illegal instruction exception is then reported by the FPCP.

6.1.12 Coprocessor-Detected Protocol Violation

All interprocessor communications in the coprocessor interface occur as standard M68000 bus cycles. A failure in this communication results in the FPCP reporting a mid-instruction exception with the coprocessor protocol violation vector number. When a protocol violation has been detected by the FPCP, the response CIR is encoded to the take mid-instruction primitive and the next read of the response CIR by the main processor terminates the dialog.

The MC68881 signals a protocol violation when unexpected accesses of the command, condition, register select, or operand CIRs occur. Coprocessor detected protocol violations occur when:

1. The MC68881 is expecting a write to the command or condition CIR, and instead an access of the register select or operand CIR occurs.
2. The MC68881 is expecting a read of the register select or operand CIR, and instead a write to the command, condition, or operand CIR occurs.
3. The MC68881 is expecting a write to the operand CIR, and instead either a write to the command or to the condition CIR or a read of the register select or of the operand CIR occurs



MC68882 signals a protocol violation when unexpected accesses of the command, condition, register select, operand, or instruction address CIRs occur. For the MC68882, coprocessor-detected protocol violations occur when:

1. The MC68882 is expecting a write to the command or condition CIR, but a read or write operation to the register select CIR or to the operand CIR or a write operation to the instruction address CIR occurs instead.
2. The MC68882 is expecting a read of the register select CIR or of the operand CIR, but a write operation to the command CIR, the condition CIR, the operand CIR, the instruction address CIR, or the register select CIR occurs instead.
3. The MC68882 is expecting a write operation to the operand CIR, but a write operation to the command CIR or to the condition CIR or a read of the register select CIR, the operand CIR, or instruction address CIR occurs instead.
4. The MC68882 is expecting a write operation to the instruction address CIR, but a write operation to the command CIR, the condition CIR, the operand CIR, or the register select CIR or a read of the operand CIR or the register select CIR occurs instead.

6

For these violations, the FPCP maps the 16-bit register select CIR onto the upper word of the 32-bit operand register. Thus, inconsistent data is read from the operand CIR, and write cycles cannot store the correct value. Of course, this is of no consequence since the protocol violation invalidates any operation being attempted by the FPCP or the main processor.

During normal operation, the FPCP synchronizes interprocessor communication by delaying the assertion of DSACKx, if necessary. However, upon detection of a protocol violation, the MC68881 always terminates the access by immediately asserting DSACKx.

Note that in certain cases resulting from serious system programming errors, an unrecoverable protocol violation may occur when using the MC68882. This particular case of the protocol violation occurs during the coprocessor interface dialog for the FMOVE and FMOVEM instructions if a read of the operand CIR occurs before the evaluate <ea> and transfer data (DR=1) or the transfer multiple coprocessor registers (DR=1) primitive is issued. In this case, the protocol violation is not reported via the take mid-instruction primitive as is the normal case. Instead, the MC68882 ignores the access completely, and it is the responsibility of the system watchdog timer to abort the access to the operand CIR by asserting the bus error signal to the main processor. The MC68020 and MC68030 cannot cause this protocol violation to occur except through misuse of the MOVES instruction.

Spurious coprocessor-detected protocol violations may also be caused by hardware timing design errors. When using an MC68020 or MC68030, a common oversight is to use a buffered address strobe ($\overline{AS}$) to the FPCP while not buffering the lower address lines supplied to the FPCP. If the $\overline{AS}$ buffer delay used is long enough, it is possible to violate FPCP specification #7. This problem is usually indicated by protocol violations during FMOVEM instructions.

A protocol violation cannot occur as a result of an access to the reserved register locations, a read of a write-only register, or a write to a read-only register (a read of a reserved or write-only register always returns a value of all ones). One exception to this rule is that a write access to the register select CIR causes a protocol violation. Reads of the save or response CIR are always valid as are writes to the restore or control CIR.

the MC68881 can request that the MPU write the instruction address CIR (by setting the 'C' bit in a primitive response), accesses of this register are neither expected or unexpected. Thus, when the MC68881 is utilized as a peripheral processor where no concurrent instruction execution occurs, requests to transfer the PC may be ignored without incurring a protocol violation. When the instruction address CIR is written by the main processor, the MC68881 updates the FPIAR with the written value without regard to "correct" protocol.

Since the MC68882 provides concurrent execution of multiple floating-point instructions, it requires program counter values to be transferred when requested to guarantee a valid FPIAR for a concurrently-executed instruction which reports an exception. Whenever the MC68882 requests the PC value, it reports a protocol violation if the main processor does not transfer the PC value by writing the instruction address CIR.

A protocol violation is the highest priority coprocessor-detected exception. It is also considered to be a fatal exception, since the MPU acknowledgment of the protocol violation exception clears any pending FPCP instruction exceptions and aborts any instruction in progress.

NOTE

To distinguish between a protocol violation detected by the MPU or the FPCP, an exception handler can read the response CIR and evaluate the returned primitive. If the protocol violation is detected by the FPCP due to an unexpected access, the operation being executed previously is aborted, and the FPCP assumes the idle state when the exception acknowledge is received. Therefore, the primitive read from the response CIR is null (CA=0). If the protocol violation is detected by the MPU due to an illegal primitive, the FPCP response CIR contains that primitive when the exception handler reads it. (Since the FPCP cannot internally generate an illegal primitive, an MPU detected protocol violation indicates a hardware failure.)

To read the response CIR in a hardware independent manner, the trap handler should use the move alternate address space (MOVES) instruction. For example, the following instruction sequence reads the response CIR of the coprocessor with CPID=1 into an MPU data register:

MOVE.B	#7,D0	Prepare the SFC register
MOVEC	D0,SFC	for a CPU space cycle. . .
MOVES.W	\$00022000,D0	Execute a "coprocessor" cycle.

6.1.13 Recovery from Exceptions

When a coprocessor-detected exception occurs, enough information is made available to the trap handler to perform the necessary corrective action and then resume execution of the program that caused the exception. Of course, in some instances, it may not be valid to resume execution of the program; recovery is not possible for protocol violations. The information available to an exception handler is described in the previous sections, and the following paragraphs describe the methods used to resume execution of a program after an exception is appropriately handled.

In all cases, the stack frame generated by the MPU in response to a coprocessor-detected exception contains a program counter value that points to the instruction to be executed



return from the exception handler. In the case of pre-instruction exceptions, the action to be executed upon return is the FPCP instruction that was attempted, but preempted by a pending exception. For mid-instruction exceptions (other than interrupts), two pointers are saved: the address of the FPCP instruction that caused the exception and the address of the instruction immediately following that FPCP instruction. Furthermore, the FPIAR contains a pointer to the FPCP instruction that caused the exception in both cases. Thus, an exception handler can always locate the instruction that caused an exception, and identify the next instruction to be executed upon return from the handler.

When the MPU executes a return from exception (RTE) instruction, it reads the stack frame from the top of the active system stack and restores that context. In the case where the stack frame was generated by an FPCP pre-instruction exception, the context that is restored is the MPU context of the pre-empted FPCP instruction. The FPCP instruction begins execution in the normal manner, with the MPU writing the coprocessor command word to the FPCP.

In the case where the RTE stack frame is generated by a coprocessor-detected mid-instruction exception, the context restore operation is slightly different. In this case, the MPU must complete execution of the instruction that was suspended by the exception. When the RTE instruction completes execution, the MPU first reads the response CIR of the FPCP to determine the next appropriate action.

6

NOTE

Since the MC68881 always finishes execution of the instruction that causes this type of exception before reporting it, the response that is returned is null (CA = 0, PF = 1), which releases the main processor to continue with the execution of the next instruction. Note that after a take mid-instruction exception primitive is returned, the main processor is not required by the MC68881 to perform a read from the response CIR before initiating the next floating-point instruction, but the MPU always performs this action when processing a mid-instruction stack frame.

An MC68881 arithmetic exception handler (i.e., a handler for any exception other than the BSUN exception) routine is not required to perform any action to clear the cause of an exception. In fact, an MC68881 arithmetic exception handler may consist of a single RTE instruction (which produces the same logical effect as disabling an exception). This is because the main processor acknowledges the exception by writing to the control CIR when the coprocessor signals an exception to the MPU, and the exception acknowledge clears any pending exceptions in the MC68881. Thus, the MC68881 arithmetic exception handler is not required to clear any status bits or read any MC68881 registers in order to prevent the reoccurrence of an exception when an RTE instruction is executed. However, an RTE instruction alone does not prevent the reoccurrence of an MC68882 exception. The MC68882 does not clear the pending floating-point exception in response to the exception acknowledge. An MC68882 arithmetic exception handler must meet certain requirements in order to clear the cause of the exception. Refer to **5.2.2 Exception Handler Code** for the MC68882 exception handler requirements. In the case of the BSUN exception handler, some action must be taken (as described in **6.1.1 Branch/Set on Unordered (BSUN)**) by the exception handler to avoid an infinitely executing loop.

For the MC68881, if an exception handler includes any FPCP instruction other than an FMOVE, an FSAVE should be the first FPCP instruction to be executed. This assures that an exception handler cannot generate any exceptions related to, or modify the context of,



program that caused the exception. For the MC68882, all exception handlers must n with an FSAVE instruction, even when they do not contain any floating-point instructions. The FPIAR value must be saved before any instruction other than an FMOVM is executed, so that the address of the instruction that caused the exception is not lost. When the exception handler completes the error recovery and is prepared to return to the suspended program, an FRESTORE is executed as the last FPCP instruction; this restores the previous context of the program that caused the exception. Refer to **5.2.2 Exception Handler Code** for other requirements of the MC68882 exception handler.

6.2 MAIN PROCESSOR DETECTED EXCEPTIONS

The following paragraphs describe exceptions that are detected by the MPU during FPCP instruction execution. Refer to the main processor user's manual for additional information on these exceptions, and the pre- and mid-instruction exception main processor stack frames.

6

6.2.1 Trap on Coprocessor Condition Instruction

The FPCP trap on condition instruction is initiated when the MPU writes a conditional predicate to the FPCP for evaluation and reads a true/false result in the FPCP response primitive. If the FPCP indicates that the condition is true, the MPU takes a post-instruction exception using the TRAPV/TRAPcc vector number.

The stack frame generated by the MPU in response to this exception contains two pointer values:

1. A pointer to the FTRAPcc instruction that caused the exception
2. A pointer to the instruction that follows the FTRAPcc (the pointer to which the processor returns if an RTE instruction is executed)

6.2.2 Illegal Instructions

The FPCP instructions consist of an operation word, a coprocessor command word (if any), and extension words (if any). The MPU detects illegal operation words, and the FPCP detects illegal command words. When the MPU detects an illegal operation word for a coprocessor instruction, it takes a pre-instruction exception using the F-line emulator vector number. Refer to **4.7 INSTRUCTION ENCODING DETAILS** for specific bit patterns that are illegal coprocessor operation words.

In addition to detecting an illegal operation word, the MPU can detect an illegal instruction even though the operation word is valid. This occurs when the addressing mode of the instruction is not valid. When the FPCP returns a primitive response to the MPU that requests a data transfer to or from the effective address, the FPCP either implicitly or explicitly indicates the valid addressing modes for an instruction. Thus, the MPU can determine that properly formed FPCP operation words and primitive responses are invalid if they specify operations that are illegal, such as writing to a nonalterable effective address.

When the MPU detects an invalid instruction in this manner, it terminates the FPCP execution of the instruction by writing an abort to the control CIR. (The MC68882 only aborts



instruction with the invalid effective address without disturbing concurrently-executed instructions. The MPU then takes a pre-instruction exception using the F-line emulator vector number. Termination of the FPCP instruction execution in this manner does not alter any visible processor or coprocessor registers or status (such as pending coprocessor exceptions). Use of the F-line emulator trap allows the operating system to emulate any extensions to the FPCP that are not supported by a specific processor.

6.2.3 Main-Processor-Detected Protocol Violations

If the MPU reads an FPCP response primitive that it interprets as an illegal primitive, it does not terminate the FPCP execution of the instruction by writing to the coprocessor interface control register. Instead, the MPU takes a mid-instruction exception using the coprocessor protocol violation vector number.

Since the FPCP never issues an illegal response primitive, this feature of the MPU serves to detect a failure of interprocessor communications. If a protocol violation is taken on an FPCP instruction, whether detected by the FPCP or the MPU, a system failure may be assumed. Refer to **6.1.12 Coprocessor-Detected Protocol Violation** for an example of how an exception handler can determine the cause of a protocol violation.

6.2.4 Trace Exceptions

To aid in program development, the MPU includes a facility to allow instruction-by-instruction tracing. In the single-step trace mode, after each instruction is executed, the MPU takes a post-instruction exception using the trace vector number. This allows a debugging program in the trace exception handler to monitor the execution of a program under test. Refer to the main processor user's manual for a complete description of the trace mode.

Many FPCP instructions can operate concurrently with MPU instructions, and defer the reporting of coprocessor detected instruction exceptions until the next FPCP instruction is dispatched by the MPU. This provides a sequential instruction execution model even though concurrent instruction execution may occur. To guarantee that pending exceptions are always reported at the same point in an instruction sequence, regardless of whether tracing is enabled, the FPCP always releases the MPU at the end of an instruction that allows concurrency before reporting the exception. This sequence is important, because the MPU (when in the trace mode) waits for an instruction to complete before proceeding.

To provide consistent reporting of exceptions, the FPCP always returns the null (CA=0, PF=1) primitive when it completes execution of an instruction that allows concurrency, and then reports a pending exception only after a write to the command or condition CIR.

The synchronization of the two devices in the trace mode is accomplished through the PF bit in the null primitive (see **7.1 CHIP-SELECT DECODE**). When the trace mode is enabled, the MPU repeatedly reads the response CIR to determine when the FPCP completes instruction execution. If the null (CA=0, IA=1, PF=0) primitive is read, then the MPU checks for pending interrupts, and if none are pending, reads the response CIR again. This process continues until the MPU receives a null (CA=0, PF=1) primitive from the FPCP, at which time it performs the trace exception processing.

der for a trace exception to be transparent to normal program execution, the trace handler routine must take certain precautions to prevent disturbing the context of the FPCP. When the main processor detects an exception, it automatically saves the most volatile portion of the current context and processes the exception immediately; thus, the trace handler routine is not required to perform any MPU context save in order for the system to operate properly. The FPCP does not operate in this manner, since it cannot initiate exception processing until the MPU attempts to execute a new floating-point instruction. Also, the context information that must be saved for the FPCP is more extensive than that of the main processor; thus, the software must perform the save only when necessary. The important consideration for a trace exception handler is that it must perform a more extensive context save for the FPCP than for the MPU (since part of the MPU context save is automatic). Also, it should not execute any FPCP instruction that may cause a pending exception to be reported, or a new exception to occur.

Because of these constraints, the first and last FPCP instructions of a trace exception handler should be the FSAVE and FRESTORE instructions, respectively. By executing the FSAVE instruction before any other floating-point instruction, the FPCP saves any pending exceptions in a state frame and then clears them internally; thus, an exception generated by the main program cannot be reported while the trace exception handler is executing. After the FSAVE instruction is executed, the FMOVEM instruction can be used to save the user-visible portion of the FPCP context. Then the trace handler is free to utilize the coprocessor as desired, without affecting the main program context. When the trace handler is ready to return to the main program, the FMOVEM instruction is used to restore the user-visible context, followed by an FRESTORE instruction to reinstate the exact context of the FPCP prior to the trace exception processing. Note that since the MPU is forced to wait until the completion of an FPCP instruction before processing a pending trace exception, the execution of the FSAVE instruction by the trace handler always results in an idle state frame being saved. The user-visible registers contain the results of the last floating-point instruction. This would not be the case if the trace exception handler were allowed to begin execution before the FPCP instruction is completed. Processors other than the MPU must implement the trace synchronization mechanism in software (by polling the PF bit) in order to assure these conditions.

6.2.5 Interrupt

When the FPCP is busy executing an instruction, it may issue a null (CA = 1, IA = 1) primitive response, which requests the MPU to continue polling the response register. (This only occurs if the FPCP requires additional services from the MPU for the current instruction.) This response also indicates to the MPU that it may sample interrupts between reads of the response CIR. If there is no interrupt pending, the MPU reads the response CIR again. If there is an interrupt pending, the MPU takes an interrupt exception using the mid-instruction stack frame. Upon exiting from the interrupt handler, the MPU repolls the FPCP response CIR to continue the suspended instruction dialog.

In the trace mode, an interrupt can temporarily break the synchronization of the MPU and the FPCP. This can occur when the MPU receives a null (CA = 0, IA = 1, PF = 0) primitive. In this case, the MPU checks for interrupts before reading the response CIR again; if an interrupt is pending, the interrupt exception is processed immediately. In response to the interrupt, the MPU saves a 10-word mid-instruction stack frame with the trace pending status saved as part of the previous context information. When the interrupt handler completes execution and performs an RTE instruction, the MPU returns to the trace pending



and reads the response CIR to determine if the previous coprocessor instruction is needed. In this manner, the exception processing for the interrupt is completely transparent to the handling of the trace exception by the MPU and FPCP pair.

If an interrupt handler for a system using an FPCP requires the use of the FPCP, or if a task switch requires that the context be saved, an FSAVE instruction should be the first floating-point instruction executed by the routine. To restore the original context, an FRESTORE must be executed by the routine before the RTE instruction. If an interrupt handler does not interact with the FPCP, no context save operations are required.

Many FPCP instructions require a fairly long time to execute, and the MPU may be forced to wait until the FPCP execution is complete before proceeding to the next instruction (either because the instruction does not allow concurrency or the main processor is in the trace mode). Normally, the MPU can only process pending interrupts when it reaches an instruction boundary, but this might adversely affect interrupt latency if it is not allowed to process interrupts while waiting on the FPCP. To reduce interrupt latency as much as possible, the FPCP always sets the interrupts allowed (IA) bit in the null (CA=1) and null (CA=0, PF=0) primitives; thus allowing interrupts to be processed while the MPU is waiting for the coprocessor to complete an operation. In fact, most FPCP instructions, regardless of their overall execution time, provide for very small interrupt latency times. The worst-case interrupt latency instruction for the FPCP is the FRESTORE with a busy state frame (see **8.3 INTERRUPT LATENCY TIMES** for more information).

6.2.6 Address and Bus Errors

Bus cycle faults may occur while processing FPCP instructions during the MPU accesses of the coprocessor interface registers, or during memory cycles run by the MPU to access instructions or data. If the MPU receives a fault while running the bus cycle which initiates an FPCP instruction (i.e., the initial write to the command or condition CIR), it assumes that no FPCP is present in the system, and takes a pre-instruction exception using the F-line emulator vector number. Thus, an MPU system may utilize software emulation of the FPCP or provide hardware floating point, and the actual configuration is transparent to the application program. If any other access to the FPCP is faulted, it is assumed that the coprocessor has failed, and the MPU takes a bus error exception.

If the MPU has a memory fault while executing an FPCP instruction, it takes an address error or bus error exception. After the fault handler corrects the fault condition, it may return and communication with the FPCP continues as if the fault had not occurred.

6.2.7 Privilege Violations

The MPU operates at one of two privilege levels: the user level or the supervisor level. The privilege level determines which operations are legal, and the S bit in the MPU status register determines the privilege level. Most programs execute at the user level where accesses are controlled, and effects on other parts of the system are limited. The operating system executes at the supervisor level, has access to all resources, and may execute all instructions; hence, it performs the overhead tasks for the user-level programs.

The FPCP FSAVE and FRESTORE instructions are privileged instructions; all others are nonprivileged. An attempt to execute the FSAVE or FRESTORE instructions while at the



6.2.8 Format Error Exceptions

When the FRESTORE instruction is executed, the FPCP checks the validity of the format word written to the restore CIR by the MPU. Refer to **6.4.2 State Frames** for information on the format word. The FPCP returns an invalid format word (\$02XX) in the restore CIR when the format word from MPU is not valid. The MPU then takes a pre-instruction exception using the format error vector number. Refer to **7.5.4.7 FORMAT EXCEPTION, FRESTORE INSTRUCTION** for further information on the FRESTORE format error exception.

When an FSAVE instruction is initiated while the FPCP is executing a previous FSAVE or FRESTORE instruction, the FPCP returns an invalid format word (\$02XX) in the save CIR. The MPU then takes a pre-instruction exception using the format error vector number. Refer to **7.5.4.6 FORMAT EXCEPTION, FSAVE INSTRUCTION** for further information on the FSAVE format error exception.

6

6.3 MC68882 EXCEPTION HANDLERS

MC68882 exception handlers can be derived by modifying existing MC68881 handlers. The required modifications are discussed in **5.2.2 Exception Handler Code**. Note that if the guidelines in the referenced text are met, the resulting MC68882 handlers can be used with no adverse effects for systems that use the MC68881. Since the MC68882 is pin-compatible and user-software-compatible with the MC68881, the exception handlers can be written to meet the system software requirements of both the MC68881 and the MC68882. When this is done, systems that only use the MC68881 at present can replace the MC68881 with the MC68882 using the same socket, without changing either applications or systems software.

6.4 CONTEXT SWITCHING

In most types of multitasking systems, it is often necessary to take control from one program and give control to another program. This requires the operating system to extract (from the FPCP) data corresponding to one program context and load the context corresponding to the next program to be executed. The information that must be exchanged is divided into two categories:

1. Programmer's model consists of data accessible by the programmer using nonprivileged instructions. This data is saved and restored using the FMOVEM instructions.
2. Internal state consists of various internal flags and registers that are vital in restoring the FPCP to the proper state. The application program need not be concerned with the internal state. These internal flags and registers are accessed by the privileged FSAVE and FRESTORE instructions.

The following paragraphs describe how this context information is manipulated.

The basic mechanism for performing a context switch on the FPCP is provided by the FSAVE and FRESTORE instructions. These instructions are a logical extension to the instruction continuation mechanism that is used by the MC68010, MC68020, and MC68030 processors to support virtual memory. The FSAVE instruction is treated much like a microcode-level interrupt to the FPCP, instructing it to suspend any operation that is being executed (at the earliest possible boundary) and make a complete copy of the internal state of the machine into memory. This is similar to the effect of the assertion of bus error to the main processor. To restore the internal state saved by the FSAVE instruction, the FRESTORE instruction is used, which is similar to the RTE instruction on the main processor.

The internal state information that is stored in memory by the FSAVE instruction contains the image of the flags and registers not visible to the user, including the address in the microprogram counter, temporary register values, and pending exception information. After the execution of an FSAVE, the FPCP enters the idle state, and any pending exceptions are cleared. To perform a complete context save, FMOVEM instructions must be used to save the user-visible portion of the machine; and then a new context may be loaded. When it is necessary to reload the context that was previously saved, these steps are reversed: first, the FMOVEM instructions load the user-visible context, followed by an FRESTORE instruction, which loads the nonuser-visible context. After the execution of the FRESTORE, the FPCP returns to the exact context that existed just before the FSAVE instruction was executed, and execution continues from that point.

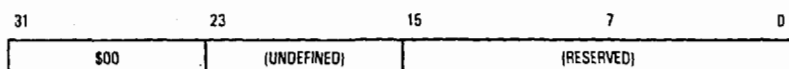
Depending on the state of the FPCP when an FSAVE instruction is executed, the format of the internal state information written to memory may be in one of three forms: idle, null, or busy. Also, the FPCP may force the MPU to wait for a short time while the internal state is prepared for the save operation. During execution of an FRESTORE instruction, the FPCP interprets the state information read from memory (and written to the restore CIR) to determine the appropriate response action. The FRESTORE is destructive in that the FPCP immediately stops any operation that it may be performing and begins to load the next context; thus there is no need for a mechanism in the FRESTORE instruction to allow the FPCP to make any service requests to the MPU. The protocol of the FSAVE and FRESTORE instructions is described in detail in a subsequent paragraph.

6.4.2 State Frames

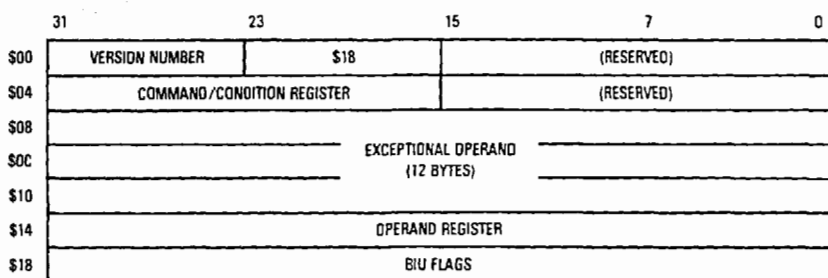
The three state frame formats that are generated by the MC68881 are shown in Figure 6-4. In all three state frames, the first long word of the frame has the same format. The least significant word of this long word is reserved for future definition by Freescale; it is included to allow long word alignment of a state frame in memory. The most significant word of the first long word (called the format word) contains the version number of the coprocessor that generated the state frame (in the most significant byte) and the size of the internal state stored in the frame (in the least significant byte). For the null state frame, the size value is undefined. Although the version number and frame size values are defined by the MC68881, the M68000 Family coprocessor interface defines the null format word which is the one format word value that must be recognized by any coprocessor as described in a subsequent paragraph.

Two of the state frame formats for the MC68882, shown in Figure 6-5, differ from the corresponding state frames for the MC68881 in two respects. First, the idle and busy state

NULL STATE FRAME



IDLE STATE FRAME



BUSY STATE FRAME

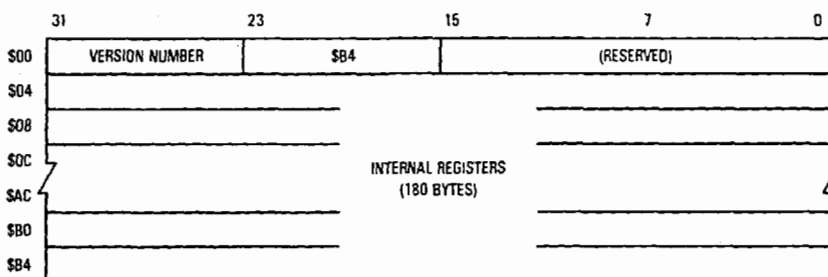
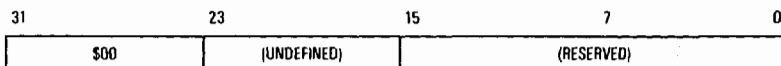


Figure 6-4. MC68881 State Frame Formats

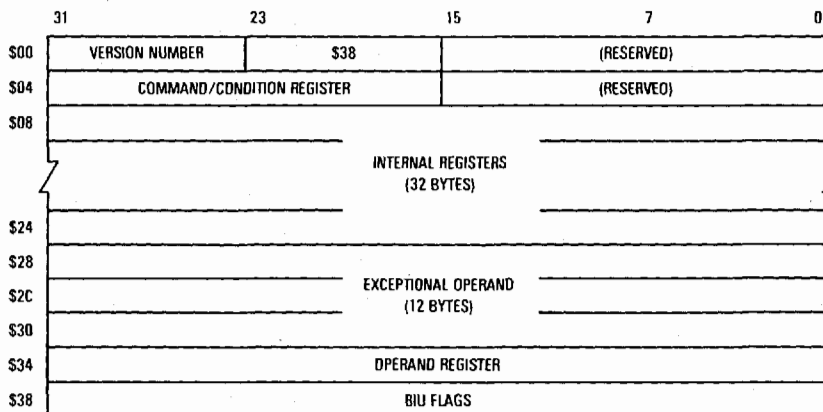
frames each contain 32 additional bytes, which store the CU internal state. Second, the saved CU internal state is saved at the top of the frame, immediately following the format word. This results in offsets to the APU information that are greater than those for corresponding data in the MC68881 state frames by \$20. The null state frame consists only of the format word in both coprocessors.

When an FSAVE instruction is executed, the format word is the first data item transferred to the MPU, and the main processor uses the size value to perform the correct address

NULL STATE FRAME



IDLE STATE FRAME



BUSY STATE FRAME

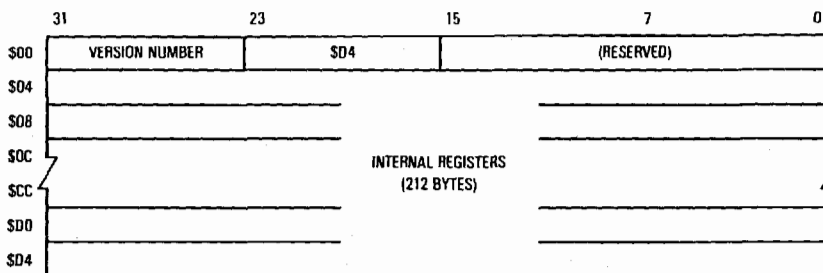


Figure 6-5. MC68882 State Frame Formats

calculations. During an FRESTORE instruction, the format word is written to the FPCP to initiate the restore operation. When this occurs, the FPCP checks the version number and frame size values for validity and signals a format exception if they are not valid for this particular device. The version number is an 8-bit value that identifies the microcode version of the FPCP, and the format of this number is defined internally by the FPCP. Future devices will use a unique combination of the version number and frame size values in order to guarantee that various revisions of the device cannot incorrectly utilize an internal state frame that is not valid for that revision.

dition to being used by the FPCP to validate a state frame before it is used in a restore operation, the format word can be used by a user program to identify the format of a state frame and the saved state of the FPCP. In the following descriptions of the three state frames, the data format within a frame is guaranteed only for those version number and frame size values given in the accompanying tables. Routines that utilize state frame information must examine the format word to correctly identify any data formats that are subject to change by Freescale.

NOTE

The state size value in the format word indicates the size (in bytes) of the FPCP internal state information. This size value does not include the format word or the reserved word.

6

6.4.2.1 NULL STATE FRAME. As shown in Figures 6-4 and 6-5, no internal state information is saved in the null state frame. Only the coprocessor version number (0) is indicated. Version number 0 is a wild card number, allowing this state frame type to be restored to a coprocessor of any version. The size value of a null state frame is not assumed to be valid during a save operation and is ignored by the FPCP during a restore operation. A restore of the null state performs the reset function with all floating-point data registers loaded with nonsignaling NaNs and with the FPCR and FPSR set to zero. A save of the null state results when no FPCP instructions have been executed since the last null state restore or hardware reset. Note that a save of a null state indicates that the FPCP programmer's model is empty, and thus does not need to be saved with a FMOVEM instruction.

6.4.2.2 IDLE STATE FRAME. As shown in Figure 6-4, 24 bytes of internal state are saved in the idle state frame for the MC68881. For the MC68882, the idle state frame consists of 56 bytes (see Figure 6-5). The format word indicates the coprocessor version number and state size (24 or 56 bytes in addition to the format word). An idle state frame is produced if an FSAVE occurs when a floating-point instruction is not being executed, or when the current instruction is in the end phase (refer to **6.4.3.5 END PHASE** for a definition of the end phase).

In addition to being used for context switching, the idle state frame contains information that is useful to most floating-point exception handlers. First, it contains the exceptional operand value, which can be evaluated by an exception handler to determine the cause of an exception. Second, it contains the BIU flag word that indicates the status of the FPCP at the time of an FSAVE instruction. For example, this information can be used by the trace exception handler in a debug monitor to display the pending exception status along with the register state of the machine.

As shown in Figure 6-4, the idle state frame for the MC68881 contains four data items: the command/condition register image, the exceptional operand, the operand register image, and the BIU flags. A reserved word is also included in order to align the state frame to a long-word boundary; it is written as \$FFFF and ignored during restore operations. The command/condition word and operand register may contain temporary information, as indicated by the BIU flags.

The idle state frame for the MC68882 contains 32 words of CU internal register and state information between the command condition register and the exceptional operand. It is otherwise identical to the idle state frame for the MC68881.

format of the BIU flag word is shown in Figure 6-6. Only the 16 most significant bits ... the BIU flag word are defined; the undefined bits are written as ones during save operations and ignored during restore operations.

The definitions of the 16 flag bits are:

- Bits 16–19 These bits contain internal state information about the CU and should not be modified.
- Bits 20–23 These bits are set when valid data is contained in the operand register image of the state frame. There is one flag bit for each byte in the 32-bit operand register image; if a bit is one, there is valid data in the corresponding byte. If a bit is zero, the data in the corresponding byte is assumed to be invalid. These bits can be used to qualify the image of the operand register and should not be modified.
- Bits 24–25 These bits contain internal state information about the CU and should not be modified.
- Bit 26 This bit indicates that the FPCP has completed any necessary operand conversions and is ready to write an operand to memory. If this bit is a zero, an operand transfer to memory is pending. This bit should not be modified.

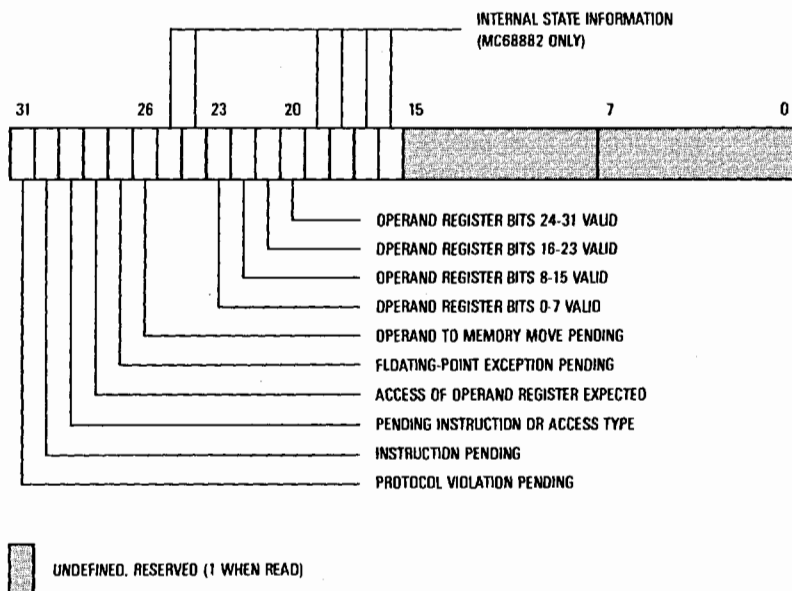


Figure 6-6. BIU Flag Format

This bit indicates that a floating-point exception is pending, which is reported when the MPU attempts to initiate the next floating-point instruction (after a FRESTORE of this state frame). If this bit is zero, an exception is pending, and the logical AND of the FPSR EXC and FPCR ENABLE bytes indicates the type of the pending exception. This bit may be read by an exception handler (particularly a trace routine) to determine the exception status of the FPCP. As described in a subsequent paragraph, a user program can modify this bit and the FPSR EXC and FPCR ENABLE byte images to create a software generated pending exception.

NOTE

This bit must be set by the exception handler immediately before an FRESTORE and RTE instruction. When this bit is not set in the exception handler, the MC68882 re-executes the handler.

6

- Bit 28** This bit indicates that the FPCP is expecting the next coprocessor interface register access to be to the operand CIR. This bit is used by the BIU as part of the protocol violation checking hardware and should not be modified. If this bit is a zero, an access of the operand CIR is pending, and the state of bit 29 determines whether the expected access is a read or write cycle. Bits 28–30 combine to define the pending operation as listed in Table 6-4.
- Bit 29** This bit defines the type of pending operand access that is expected or the type of pending operation that is saved in the command/condition register image. This bit should not be modified. Bits 28–30 combine to define the pending operation as listed in Table 6-4.
- Bit 30** This bit indicates that the FPCP has received a new command word or conditional predicate from the MPU, but has not been able to begin execution of that operation. If this bit is zero, the command word or conditional predicate that was received is contained in the command/condition register images of the state frame. This bit should not be modified. Bits 28–30 combine to define the pending operation as listed in Table 6-4.
- Bit 31** This bit indicates that a protocol violation has been detected by the FPCP, and the MPU has not responded with an exception acknowledge or abort operation. If this bit is a one, a protocol violation is pending. This bit should not be modified.

NOTE

The formats of the idle state frame and the BIU flags shown are for the initial production versions of the FPCP; this format is identified by the format word values (\$1F18 and \$3F18 for the MC68881, and \$1F38 for the MC68882). Freescale reserves the right to utilize different state frame formats and format word values to support future revisions to the FPCP.

Table 6-4. BIU Flag Bit Definitions

30	29	28	Definition
0	0	0	(Undefined, Reserved)
0	0	1	Conditional Instruction Pending
0	1	0	(Undefined, Reserved)
0	1	1	General Instruction Pending
1	0	0	Write of Operand CIR Pending
1	0	1	(Undefined, Reserved)
1	1	0	Read of Operand CIR Pending
1	1	1	No Pending Instruction or Operand CIR Access

The only bit in the BIU flag word that can be modified by software is bit 27, the exception pending bit. If this bit is zero, an exception is pending and may be cleared by changing it to a one. Alternatively, the type of the pending exception can be changed by modifying the FPSR EXC byte and/or the FPCR ENABLE byte before executing an FRESTORE. Finally, if the pending exception bit is one (indicating that no exception is pending), it can be changed to make an exception pending; the type of exception pending is defined by the FPSR EXC and FPCR ENABLE bytes. In all of these cases, the change in the exception status takes effect when the state frame is utilized by an FRESTORE instruction.

6

The exception pending bit (referred to as EXC PEND) in the BIU flag word is the image of the exception pending signal internal to the FPCP. Normally, EXC PEND is negated by the FPCP execution unit when an instruction (other than an FMOVEM, FMOVE control register, FSAVE, FRESTORE) begins execution, and is asserted if an exception occurs during the instruction. The bus interface unit uses EXC PEND to determine the primitive response that is encoded in the response CIR after a write to the command or condition CIRs, or after the completion of the transfer of a floating-point operand to memory. If EXC PEND is true when an attempt is made to initiate an FPCP instruction (other than an FMOVEM, FMOVE control register, FSAVE, or FRESTORE), the response CIR is encoded to the take pre-instruction exception primitive (or the take mid-instruction primitive when the instruction in the CU is reporting an exception caused by the instruction in the APU); otherwise, the dialog for the instruction is started. If EXC PEND is true at the end of the move of a floating-point operand to memory, the response CIR is encoded to the take mid-instruction exception primitive; otherwise it is encoded to the null (CA = 0, PF = 1) primitive. The vector number that is encoded in the take exception primitive is determined by the state of the FPSR EXC and FPCR ENABLE bytes and corresponds to the highest priority exception that is enabled. The MPU responds to the take exception primitive by writing an exception acknowledge to the control CIR. When the MC68881 detects the exception acknowledge, it clears EXC PEND. However, the MC68882 does not clear the EXC PEND bit. It is the responsibility of the exception handler to clear EXC PEND, using the instructions listed in **5.2.2 Exception Handler Code**.

With this understanding of how EXC PEND (and its image in the BIU flag word) affects the operation of the FPCP, a programmer can make exceptions pending in the FPCP under software control. Or, conversely, a pending exception type may be changed or cleared if necessary.

6.4.2.3 BUSY STATE FRAME. As shown in Figure 6-4, 180 bytes of internal state are saved in the MC68881 busy state frame. The format word indicates the coprocessor version

er and state size (180 bytes). The busy state is produced if an FSAVE occurs when a floating-point instruction is in the initial or middle phase. Due to the volatile nature of the internal state during calculation, this state frame does not contain any information useful to applications programs, and the frame should not be modified in any way.

The MC68882 busy state frame contains 212 bytes, including 32 bytes of CU internal state information (refer to Figure 6-5). The format word contains the coprocessor version number and the state size. Otherwise the MC68882 busy state frame is identical to the busy state frame of the MC68881.

6.4.3 FSAVE Protocol

Table 6-5 lists five possible phases of the execution of a floating-point instruction that can apply at the time an FSAVE instruction is executed. For each phase, the table shows the response time and the state frame type.

Table 6-5. MC68881/MC68882 Responses to Save Command

Phase Name	Response Time	State Frame Type
Reset	Immediate	Null
Idle	Immediate	Idle
Initial	Immediate	Busy
Middle	Periodic	Busy
End	Delayed	Idle

When the MPU decodes an FSAVE instruction, it attempts to initiate a save operation in the FPCP by reading from the save CIR. If the FPCP is ready to perform the save, it responds with a valid state frame format word. The format word informs the MPU that the coprocessor is ready to transfer the state frame and also what size frame is to be saved. If the FPCP is not ready to begin the transfer of the state frame, it returns the come-again format word, forcing the MPU to wait. When the MPU receives the come-again format word, it checks for pending interrupts and processes them if necessary. Otherwise, it repeatedly reads the save CIR until a format word other than come again is returned. When the FPCP receives a valid format word, it reads the number of bytes indicated by the format word, four bytes at a time, from the operand CIR and writes them to memory.

The FPCP always returns one of five format words in the save CIR. Table 6-6 shows the five format word values and their meanings. In this table, the version number of the idle and busy format words corresponds to the version number of the initial production versions of the MC68881; future revisions of the device will utilize different version numbers to identify unique state frame formats. If the format of the idle or busy state frame of a future version of the FPCP differs from that of versions \$1F and \$3F for the MC68881 or \$1F for the MC68882, Freescale will provide the new format information when the new version is available.

The come-again format word is returned by the FPCP to force the MPU to wait, as previously described. When the FPCP is ready to complete a save operation, it returns one of the other valid format words (null, idle, or busy) to the main processor and then transfers the

Table 6-6. MC68881/MC68882 Format Word Definitions

Format Word	Definition and Frame Size
\$00xx*	Null State
\$01xx*	Come Again
\$02xx*	Illegal, Format Error
\$vv18**	Idle State (MC68881)
\$vvB4**	Busy State (MC68881)
\$vv38**	Idle State (MC68882)
\$vvD4**	Busy State (MC68882)

*The frame size byte for these format words is undefined for the M68000 Family coprocessor interface. The value encoded by each version is consistent; however, different versions are not guaranteed to use the same values.

**Each different version encodes a unique version number in "vv" while using the same frame size value.

appropriate state frame. The only time that the FPCP uses the illegal format word is when a read of the save CIR occurs while the FPCP is performing a state save or state restore. Normally, this only occurs when the execution of an FSAVE or FRESTORE instruction is suspended (e.g., due to a page fault during the save or restore operation) and an attempt is made to execute a new FSAVE instruction. If this happens, the illegal format word is returned to cause a format exception to be taken by the main processor. When the MPU receives the illegal format word, it writes an abort to the control CIR and initiates exception processing. In this case, the format error handler routine examines the instruction that was being executed when the format error occurred and can determine whether the second FSAVE instruction failed due to "nesting" of save or restore operations. Such an error is considered to be a catastrophic system error since the FPCP context is lost and cannot be recovered.

When the MPU receives an idle or busy format word, the bytes in the frame (four bytes at a time) are transferred from the operand CIR to memory. First, the format word is written to memory at the evaluated effective address. For the predecrement addressing mode, the value of the specified address register is saved in a temporary register, the size of the state frame is subtracted from the address register, and the format word is pushed to that address. (Thus, the required stack space is allocated before the save operation is started.) The state frame is then filled, from higher addresses to lower addresses, using the temporary register as a pointer. For the control alterable addressing modes, the format word is written to the specified address; then the address of the last word of the frame is calculated (in a temporary register) and the frame is filled from higher addresses to lower addresses. After the last byte of the state frame is written to memory, the FPCP is in the idle state with no pending exceptions, and the MPU executes the next instruction (it does not read the save or response CIR after the save operation).

The following paragraphs describe the response of the FPCP to an FSAVE instruction for the various phases of instruction execution.

6.4.3.1 RESET PHASE. In this phase, no FPCP instructions have been executed since the last hardware reset or FRESTORE of a null state frame. When the FPCP is in this state and an FSAVE is executed, a null format word is returned immediately.

2 IDLE PHASE. In this phase, the FPCP is not executing an instruction, but at least instruction has been executed since the last hardware reset or FRESTORE of a null state frame. When the MC68881 is in this state and an FSAVE is executed, an idle format word is returned immediately, and an idle state frame is stored.

6.4.3.3 INITIAL PHASE. In this phase, the FPCP is acquiring instruction and operand words from the MPU. In a virtual memory system, a memory fault can occur during this phase due to an attempt to access an operand that is not resident in main memory. In this case, the MPU traps to a fault handler to initiate a transfer from secondary storage, typically involving one or more disk accesses. After initiating the transfer, the operating system usually switches the main processor and coprocessor(s) to another program, thus necessitating a save of the coprocessor state and restoration of the state of the coprocessor relative to the next program. To facilitate saving and restoring the coprocessor, the FPCP responds immediately to a save command during the initial phase by storing a busy state frame.

6.4.3.4 MIDDLE PHASE. The middle phase occurs only in FPCP instructions that take significant processing time (i.e., remainder, transcendental functions, and BCD conversions). During this phase, the internal microcode sequence of the FPCP provides for periodic checkpoints to determine if the MPU has issued a save command. If the MPU initiates a save command to the FPCP between check points, the FPCP sets an internal flag to denote the receipt of the command and returns a come-again format word to the MPU. The MPU repeatedly reads the save CIR until it receives a valid format word. The FPCP continues internal processing up to the next checkpoint, at which time processing stops, and the next read of the save CIR acquires the appropriate format word to start the save operation. At this point, the save command proceeds to completion, and the FPCP supplies a busy state frame.

6.4.3.5 END PHASE. This phase begins when the FPCP is almost finished with a long instruction. The length of the end phase is approximately equal to the amount of time required to perform a save of a busy state frame. When the FPCP reaches the end phase, it takes less time to complete execution of the instruction and save an idle frame than to immediately save a busy state. During this phase, the FPCP uses the come-again format word to force the MPU to wait for the completion of the instruction, and then saves an idle state frame.

Note that most of the FPCP instructions proceed directly from the initial phase to the end phase, and thus, most state frames generated by the FPCP are idle frames.

6.4.4 FRESTORE Protocol

When the MPU decodes an FRESTORE instruction, it evaluates the effective address to locate the format word for the state frame, and writes that format word to the restore CIR of the FPCP. In response to this write cycle, the FPCP aborts any operation that may be in progress and prepares to load a new internal state. The format word that is written to the restore CIR is checked for validity (it must be a null, idle, or busy format word with a version number that matches that of the specified device) before the restore operation begins. After the MPU writes the format word to the FPCP, it then reads the restore CIR to verify



If the format word is valid. If the format word is valid, the FPCP returns the same format word that was written; if the format word is not valid, an illegal format word (\$02xx) is returned. If the format word is successfully verified, the MPU begins to transfer the state frame, four bytes at a time, from memory to the FPCP operand CIR.

When transferring the state frame from memory to the FPCP, the MPU first transfers the format word and, after it is verified, transfers the remainder of the state frame. The order of transfer is the same for both the postincrement and the control addressing modes. The long word at the lowest address is transferred first, followed by the long words at successively higher addresses. For the postincrement addressing mode, the specified address register is not updated by the MPU until the entire frame has been successfully transferred. Thus, a fault during an FRESTORE instruction generates a stack frame that does not overwrite any part of the FPCP state frame.

After the entire state frame has been transferred to the FPCP, the MPU continues with the execution of the next instruction (it does not read the response CIR). If an exception related to the FPCP caused the suspension of the task earlier, an RTE instruction is eventually executed to return to the original context. Depending on the exception type, the RTE may re-establish the MCU/coprocessor protocol of the suspended operation or begin the execution of a new FPCP instruction.

6.4.5 Context Switching Summary

To perform a complete context save or restore operation, three FPCP instructions are required. First, the FSAVE and FRESTORE instructions are used to transfer the nonuser-visible portion of the machine state between the FPCP and memory. Second, the FMOVEM instruction may be used to transfer the user-visible portion of the machine, including the floating-point data and control registers.

An important aspect of the FMOVEM instruction is that it cannot cause an exception or report a pending exception; thus the context of the FPCP, including pending exceptions, can be saved and restored in a manner that is completely transparent to a user program. Note that if an FSAVE instruction stores a null state frame, the floating-point data and control registers are reset to their default states, and the FMOVEM instructions are not needed. Figure 6-7 illustrates the manner in which a full context switch might be performed.

INSTRUCTION SEQUENCE TO STORE THE PREVIOUS CONTEXT

FSAVE	—(An)	SAVE MC68881 STATE FRAME
TST.B	(An)	CHECK FOR A NULL FRAME
BEQ	NULL_SV	SKIP PROGRAMMER'S MODEL; SAVE IF NULL
FMOVM	FP0-FP7, —(An)	ELSE, SAVE DATA REGISTERS
FMOVM	FPCR/FPSR/FPIAR, —(An)	AND SAVE CONTROL REGISTERS
MOVE.L	#—1, —(An)	PLACE NOT-NULL FLAG ON STACK
NULL_SV		

INSTRUCTION SEQUENCE TO LOAD THE NEXT CONTEXT

TST.B	(An)	CHECK FOR NULL FRAME OR NOT-NULL FLAG
BEQ	NULL_RST	SKIP PROGRAMMER'S MODEL; RESTORE IF NULL
ADDQ.L	#4, An	ELSE, THROW AWAY THE NOT-NULL FLAG
FMOVM	(An) + , FPCR/FPSR/FPIAR	RESTORE THE CONTROL REGISTERS
FMOVM	(An) + , FP0-FP7	RESTORE THE DATA REGISTERS
FRESTORE	(An) +	RESTORE THE FPCP STATE FRAME
NULL_RST		

Figure 6-7. Full Context Save/Restore Instruction Sequences